TEAM UW SYFI: FULL-AGENT TRACK WRITE-UP FLASHINFER AI KERNEL GENERATION CONTEST @ MLSYS 2026

Keisuke Kamahori [1] **Steven Gao** [1] **Shihang Li** [1] **Wei Shen** [1] **Yile Gu** [1]

ABSTRACT

We describe Team UW SyFI's submission to the full-agent track of the FlashInfer AI Kernel Generation Contest at MLSys 2026, in which we placed first in the GDN track and third in the DSA track. We built a fully autonomous kernel-generation pipeline that drives a coding agent (Claude Code with Claude Opus 4.7, or Codex CLI with GPT 5.5) in headless mode through a two-phase refinement loop. The first phase takes the PyTorch reference and emits a Triton kernel; the second seeds the best Triton source as a hint and emits a C++/CUDA kernel. Every round is gated by a real flashinfer-bench evaluation whose per-workload status, speedup, and error tail feed back into the next round's prompt. All five final CUDA kernels were emitted by this loop with no human edits to the generated source.

1 SOLUTION OVERVIEW

Team UW SyFI placed first in the GDN track and third in the DSA track of the full-agent contest. The submission repository is at `https://github.com/kamahori/mlsys-contest-syfi-fully-agent`.

We built a fully autonomous kernel-generation pipeline that drives a coding agent in headless mode. We use two interchangeable agent backends, Claude Code with Claude Opus 4.7 and Codex CLI with GPT 5.5, and run each through the same outer loop. For every contest definition the pipeline runs a two-phase refinement loop: phase one takes the PyTorch reference and emits a Triton kernel over N rounds, and phase two seeds the best Triton source as a hint and emits a C++/CUDA kernel over a further N rounds. Every round is gated by a real flashinfer-bench evaluation that checks correctness and measures speedup on every workload of the chosen definition; the per-workload status, speedup, and error tail are then fed back to the agent as the prompt for the next round, with no human edits to the generated source. The five definitions covered by the pipeline are listed in Table 1, and all five final kernels were emitted by the agent loop and submitted under the tvm-ffi binding in destination-passing style.

[1]University of Washington, Seattle, Washington, USA. Correspondence to: Keisuke Kamahori <kamahori@cs.washington.edu>.

2 KERNEL DESIGN AND OPTIMIZATIONS

This section summarizes the techniques the agent converged on for each kernel. All targets are sm_100 (Blackwell), use bf16 storage with fp32 accumulators, and are checked at the contest tolerance `rtol = atol = 1e-2`.

2.1 moe: FP8 block-scale fused MoE

The kernel is a single CUDA translation unit implementing an eleven-stage pipeline written from scratch. The stages, in order, are routing, permutation counting, prefix-sum offset computation, permutation fill, hidden-state dequantization, gather, weight dequantization, the bf16 GEMM, SwiGLU, final accumulation, and a final cast. Routing is the DeepSeek-V3 no-aux variant of group top-k, with an argmax-with-lane-tiebreak so that group scores remain deterministic under ties. Both GEMMs use `nvcuda::wmma` bf16 tensor cores after FP8 block-scale dequantization, and the final scatter is an fp32 atomic add. The contest-fixed constants (`H=7168`, `I=2048`, `BS=128`, `E_LOCAL=32`, `E_GLOBAL=256`, `TOP_K=8`, `N_GROUP=8`, `TOPK_GROUP=4`) are baked in as `constexpr`, allowing the agent to specialize tile shapes throughout the kernel.

2.2 gdn_prefill: chunked WY-form delta rule

This bf16 tensor-core kernel processes chunks of $C = 16$ tokens. Within a chunk the agent rewrites the gated delta rule into the un-gated WY form

$$\hat{s}_t = \hat{s}_0 + \sum_{s \le t} \beta_s \, k_s \, u_s^{\mathsf{T}},$$

Table 1. Submitted kernels (full-agent track).

Definition	Lang	Entry point
`moe_fp8_block_scale_ds_routing_topk8_ng8_kg4_e32_h7168_i2048`	CUDA	`moe.cu::run`
`gdn_prefill_qk4_v8_d128_k_last`	CUDA	`gdn_prefill.cu::run_prefill`
`gdn_decode_qk4_v8_d128_k_last`	CUDA	`gdn_decode.cu::run_decode`
`dsa_topk_indexer_fp8_h64_d128_topk2048_ps64`	CUDA	`dsa_topk_indexer.cu::run`
`dsa_sparse_attention_h16_ckv512_kpe64_topk2048_ps64`	CUDA	`dsa_sparse_attention.cu::run`

where u is the residual of the triangular solve $(I + L)\, u = \tilde{v} - K\, \hat{s}_0$. Each thread block uses four warps (128 threads) and parallelizes the four intra-chunk matmuls (KS, QS, KK, QK) one per warp; attention and the $K^{\mathsf{T}} \cdot U$ state update are split by output tile. Double-buffered `cp.async.cg` loads of Q, K, and V overlap with compute. The shared-memory row stride is padded to `K_PAD = K_DIM + 16 = 144` bf16 elements to break the 128-stride bank conflict that otherwise serializes `ldmatrix`.

2.3 gdn_decode: single-warp register-tiled state

The grid has $B \cdot H_v \cdot V/V_{\text{TILE}} = B \cdot 8 \cdot 16$ blocks, each consisting of a single warp. Each warp owns 8 v-rows, and each thread holds `K_LOCAL = K/32 = 4` K-slots, so the per-warp state is 32 fp32 values held in registers. K-reductions are warp shuffles, and old/new state and inputs are loaded with `float4` vectorization. The output is derived from `qS(old)`, `kS(old)`, and the `qk` dot product to avoid a second pass over the updated state.

The submitted `gdn_decode.cu` launches the kernel through NVRTC and the CUDA Driver API rather than the Runtime API. The agent embeds the kernel source as a string and `dlopens` `libnvrtc.so.12` and `libcuda.so.1` to compile to PTX and launch via `cuLaunchKernel`. This sidesteps a build environment in which the translation unit links against `libcudart.so.13` while the host driver is R570 with CUDA 12.8, so that every Runtime API call (including the <<<>>> launch syntax) returns error 35. NVRTC and the Driver API sit below libcudart and have no runtime/driver version check.

2.4 dsa_topk_indexer: paged FP8 with register-blocked scoring

The kernel tracks the contest's per-page memory layout, which is $64 \times 128 = 8192$ B of fp8 values followed by $64 \times 4 = 256$ B of fp32 scales (with stride `HEAD_DIM_SF = 132`). It processes Q in chunks of `Q_CHUNK = 8` heads, so eight chunks cover all 64, and tiles the head dimension by `D_CHUNK = 16` (eight inner chunks). Per-page K dequantization is reduced from `Q_CHUNK · HEAD_DIM` to `HEAD_DIM` per inner tile, an eight-fold saving, by caching 16 K-values across all Q heads for the chunk.

2.5 dsa_sparse_attention: flash-decoding with WMMA

Each CTA holds one tile and runs 256 threads (eight warps). It reads `BLOCK_N` KV tokens, computes $[H{=}16, D_{\text{TOT}}{=}576] \times [D_{\text{TOT}}, \text{BLOCK\_N}]^{\mathsf{T}}$ on the WMMA tensor cores, applies softmax one row at a time, and then computes $[H, \text{BLOCK\_N}] \times [\text{BLOCK\_N}, D_C{=}512]$ for the PV multiplication. The split count `K_SPLITS` is dispatched on the host based on `Nt`: a small `Nt` receives more splits in order to fill the machine, while a large `Nt` receives fewer splits so that each CTA carries more work and partial-O traffic stays bounded. The shared-memory stride is padded to `D_TOT_PAD = D_TOT + 8 = 584`, shifting the per-row bank offset by four to break the $D_{\text{TOT}} \times 2\,\text{B} = 1152\,\text{B}$ periodicity that would otherwise serialize `ldmatrix`.

3 Tools and Languages

Generation. We use two interchangeable agent backends. Claude Code runs Claude Opus 4.7 (`claude-opus-4-7`) via the `claude` CLI in headless mode, with `-p`, `--output-format json`, and `--permission-mode bypassPermissions`, and resumes sessions across rounds with `--resume`. Codex CLI runs GPT 5.5 in headless mode, configured against the same workdir, skills directory, and MCP server. Each agent runs the same outer loop and is graded by the same harness, so we can compare generation quality on identical prompts.

Output languages. Phase one emits Triton, and phase two emits C++/CUDA seeded with the best phase-one Triton source as a hint. Both languages target the contest harness via `pack_solution_from_files` with a `BuildSpec`, and the submitted CUDA kernels bind through tvm-ffi.

Profiling. We wrote a small custom MCP server, ncu-mcp (under `mcps/ncu-mcp/`), that exposes `list_sections_and_sets`, `profile`, and `read_report_details` tools. We found that piping the output of ncu, which could be quite verbose, unnecessarily bloats context length and could lead to worse

performance. To resolve this, we created an outer loop where an agent reflects on the traces of rollouts in natural language and iterates on our initial ncu design to strike a balance between brevity and detail. The outer loop converged on a design that provides MCP tools to read different sections of the report based on the optimization needs of the agent.

Skills. Sixteen skills under `.claude/skills/` provide on-demand domain hints (Claude Code consumes them natively, and the same content is surfaced to Codex CLI through its prompt-injection layer). The skills are: cublas-cudnn, cuda-debugging, cuda-graphs, cuda-toolkit, cutlass-triton, gpu-benchmarking, gpu-memory-analysis, hip-rocm, nccl-communication, nsight-profiler, nvenc-nvdec, opencl-runtime, parallel-patterns, stencil-convolution, tensorrt-optimization, unified-memory, vulkan-compute, and warp-primitives, adopted from https://github.com/a5c-ai/babysitter/tree/main/library/specializations/gpu-programming/skills. The driver curates two subsets, one Triton-focused and one CUDA-focused, and lists them in the round's prompt as hints.

Evaluation. The harness's `Benchmark` object is run either locally or fanned out across N Modal B200 containers via `.starmap()`. The driver's default Modal configuration is one solution sharded across N containers over the workload list, giving a wall time of approximately $(W/N) \cdot (R + S)$ + container_start, where R is the per-workload reference baseline timing and S is the per-workload solution timing. A per-definition reference-latency cache (`runs/_ref_cache/<def>.json`) skips the expensive baseline timing loop once every workload has a cached value, which dominates wall time when the reference is a slow Python loop (for example, gdn_prefill).

Driver language. The driver is written in Python (`run_pipeline.py`), runs in a uv-managed virtual environment, and uses `subprocess.run` to invoke either the `claude` CLI or the `codex` CLI depending on the selected backend.

4 Agent Architecture and Workflow

4.1 One round

For each language $L \in \{\text{triton}, \text{cuda}\}$, a single round proceeds in four steps.

In the first step, the driver stages context into the per-phase workdir: `DEFINITION.md`, `definition.json` (the full schema), and `reference.py` (the verbatim PyTorch reference, copied from `references/<track>.py`).

In the second step, the driver builds the prompt. In round zero the prompt states the contest goal, gives the IO contract for L (DPS arity, binding rules for tvm-ffi or pybind11, target architectures), points the agent at the staged files, and lists the curated skills together with the `mcp__ncu-mcp__*` tools the agent may call. In rounds one through N the prompt is a refinement prompt containing the CORRECT or INCORRECT verdict, the `passed/total` workload count, the mean, median, and minimum speedups, the `max_abs_err` and `max_rel_err` values, the first eight per-workload entries with their log tails, and a tuning-levers paragraph specialized to L. If a prior best correct version exists, the agent is told to make targeted edits rather than a full rewrite.

In the third step, the driver runs the agent CLI in the workdir, either `claude -p` for Claude Code or `codex` for Codex CLI, with `--mcp-config .mcp.json --strict-mcp-config` (or the equivalent Codex flag), optionally resuming the prior session. The default per-call timeout is ninety minutes.

In the fourth step, the driver packs and evaluates the solution. It calls `flashinfer_bench.agents.pack_solution_from_files` with a `BuildSpec` matching the kernel's `config.toml`, dumps the resulting `solution.json`, and runs `Benchmark.run_all` either locally or on Modal B200 (sharded across containers via `.starmap` when `--modal-shards` is greater than one). Per-workload statuses are reduced to an `EvalResult` (correct flag, num_passed, mean, median, and minimum speedup, and the maximum errors). The round's source tree is archived to `round_NN_src/` and, if it improves on the best correct mean speedup so far, is also copied to `best_src/`; the per-round eval results are appended to `history.json`.

4.2 Two phases

Phase one uses the Triton-focused skill subset (cutlass-triton, parallel-patterns, warp-primitives, gpu-memory-analysis, gpu-benchmarking, and nsight-profiler). Phase two uses the CUDA-focused subset (the above plus cuda-toolkit, cuda-debugging, and cublas-cudnn) and is seeded with the best phase-one Triton source as a hint that the agent may study.

4.3 Correctness and performance verification

Every round is verified end-to-end against the contest harness with the exact contest tolerances (`rtol = atol = 1e-2`), the contest harness's destination-passing-style adapter, and the contest workload set drawn from `TraceSet`. There is no separate agent-side test; verification is the contest's verification.

Listing 1. Launching a single track of the full-agent pipeline.

```
uv run python run_pipeline.py \
    --definition
    gdn_decode_qk4_v8_d128_k_last \
    --workdir runs/gdn_decode \
    --triton-entry
    gdn_decode.py::run_decode \
    --cuda-entry
    gdn_decode.cu::run_decode \
    --binding tvm-ffi \
    --triton-iters 4 --cuda-iters 6
```

4.4 Failure modes the loop handles automatically

The loop handles three classes of failure that would otherwise burn agent calls. First, when the harness error string contains markers such as "CUDA driver version is insufficient", the driver aborts the phase rather than asking the agent to fix something the kernel cannot fix; this is what motivated the NVRTC and Driver API path in `gdn_decode.cu`. Second, the per-definition latency cache prevents the expensive baseline loop from running more than once per workload across the whole campaign. Third, per-workload results from N shards are merged client-side, and the reference is paid once per shard, so wall time is sub-linear in N (roughly 3 to $3.5\times$ speedup at $N = 4$ in our runs, with diminishing returns past $N = 8$).

5 REPRODUCIBILITY

The full pipeline lives under `full-agent-pipeline/` in the submission repository. Setup requires two `uv sync` invocations, one in `full-agent-pipeline/` (which installs flashinfer-bench, torch, triton, and modal) and one in `mcps/ncu-mcp/` for the ncu MCP server, followed by a one-time `git lfs` clone of `huggingface.co/datasets/flashinfer-ai/mlsys26-contest` (about 1.8 GB).

A single track is launched with the command shown in Listing 1. The arguments specify the contest definition, the per-track workdir, the Triton and CUDA entry points, the binding (tvm-ffi), and the per-phase round counts.

Per-track output lives under `runs/<track>/` as two parallel sub-trees `triton/` and `cuda/` of identical shape, plus a top-level `summary.json`. Each phase subtree contains the staged agent context (`DEFINITION.md`, `definition.json`, `reference.py`), the live solution source under `solution/<lang>/`, per-round source archives `round_NN_src/`, the best correct source by mean speedup under `best_src/`, the bench logs and packed `solution.json` under `logs_round_NN/`, and `history.json` with per-round eval results.

The contest README in `full-agent-pipeline/README.md` provides the per-track flag tables, a one-liner to run all five tracks sequentially, and the Modal B200 setup instructions. Reproducing the pipeline requires an NVIDIA GPU with `ncu` on the PATH (or `--use-modal` for benchmarking on B200), the `claude` CLI and/or the `codex` CLI authenticated, and the `uv` and `git-lfs` tooling.